AEC1 · Cómo está resuelta (Python)

Paradigmas de la Programación · Angel Panadero Rodriguez · Python 3

Guía para entender y replicar a mano el fichero `ejemplo.py`. De cada ejercicio tienes la idea en palabras sencillas, el código y una **traza**: cómo se va ejecutando paso a paso con el ejemplo del enunciado. Las respuestas teóricas completas están en los comentarios del propio fichero. Tu versión la entregas como `angel_panadero_rodriguez_AEC1.py`.

> **¿Se puede entregar en Python?** Sí, siempre que el enfoque sea funcional. Lo dicen el enunciado y el profesor:
>
> > *«Es posible usar otros lenguajes (Python, c#, Java, Scala, Swift, Kotlin, f#, Rust,...) siempre y cuando la aproximación sea funcional.» (AEC1.pdf, apartado 1)*
>
> > *«Se puede usar Java, se puede usar Python [...] No tiene sentido que uses Java y después estés usando de repente mutación de variables.» (Clase UD2, min. 38)*

1. Cómo ejecutarlo

Python 3.12 ya está instalado. Desde la carpeta AEC1:

```
python ejemplo.py
```

Muestra el resultado de cada ejemplo y, en el ejercicio 9, te pide un número. Para probar funciones sueltas, usa `python -i ejemplo.py`: ejecuta el fichero y te deja en la consola de Python con todas las funciones cargadas. Escribe por ejemplo `longitud_lista([1, 2])` y pulsa Intro. Sales con `exit()`.

2. El Python que necesitas

En Python los bloques se marcan con la **sangría** (4 espacios), no con paréntesis. Con esto tienes todo lo que usa el fichero. La última columna es su equivalente en LISP, por si te ayuda a relacionarlo con lo visto en clase:

Python	Qué hace	En LISP
`def f(a, b):` `  return x`	Define una función. Devuelve lo que pongas en `return`.	`(defun f (a b) …)`
`lista[0]`	Primer elemento: `[1, 2, 3][0]` → 1.	`(car l)`
`lista[1:]`	Lista **nueva** con el resto: `[1, 2, 3][1:]` → [2, 3]. La original no cambia.	`(cdr l)`
`not lista`	True si la lista está vacía. Es la condición de los casos base (sin usar `len`).	`(null l)`
`[x] + lista`	Lista nueva con x delante: `[1] + [2, 3]` → [1, 2, 3].	`(cons x l)`
`(a, b)`	Tupla: `(1, 2)`. Como una lista, pero no se puede modificar.	`(list a b)`

`if c:`
`  return x`
`return y`
	Si c es cierto, devuelve x. Un `return` termina la función, así que lo de abajo solo se ejecuta si c es falso: no hace falta `else`.	`(if c x y)`

`min(a, b) / max(a, b)`	El menor / el mayor de dos números: `min(9, 3)` → 3.	`(min a b)`

Python	Qué hace	En LISP
`f  y  f()`	Sin paréntesis pasas la función como dato; con paréntesis la ejecutas.	`#'f y (funcall f)`
`lambda x: 2 * x`	Función sin nombre: «dado x, devuelve x por 2».	`(lambda (x) (* 2 x))`
`list(map(f, lista))`	Aplica f a cada elemento. `map` da un iterador y `list(...)` lo convierte en lista.	`(mapcar f l)`
`reduce(f, lista)`	Junta todos los elementos en un valor: `reduce(lambda a, b: a + b, [3, 2, 4])` → 9. Hay que importarlo: `from functools import reduce`.	`(reduce f l)`
`print(x)`	Escribe en pantalla y salta de línea.	`(format t "…~%")`
`input(texto) / int(t)`	Pregunta y devuelve lo escrito como texto / convierte texto en número.	`(read)`
`# comentario / 'a'`	Comentario / cadena de un carácter (Python no tiene tipo carácter).	`; / #\a`

Lo que no debes usar (rompe el enfoque funcional o una restricción del enunciado):

- `for` y `while`. Tampoco las comprensiones `[x * 2 for x in lista]`: son funcionales, pero llevan `for` y pueden parecer un bucle.
- `len` (prohibido expresamente en el ejercicio 2).
- Modificar listas o variables: `append`, `+=`, `lista[i] = …`, `x = x + 1`.

3. La plantilla que se repite

Casi todos los ejercicios son recursión sobre listas: resuelves el primer elemento (`lista[0]`) y llamas a la misma función con el resto (`lista[1:]`) hasta llegar al caso base. Antes de escribir, responde a dos preguntas: **¿cuándo paro y qué devuelvo?** (caso base) y **¿cómo junto el primer elemento con el resultado del resto?** (caso recursivo).

```
def f(lista):
    if not lista:                          # ¿paro?
        return valor_caso_base             # caso base
    return juntar(lista[0], f(lista[1:]))  # caso recursivo
```

Truco de lectura: `not lista[1:]` significa «el resto está vacío», es decir, **solo queda un elemento**. Así queda cada ejercicio con esta plantilla:

Ejercicio	¿Cuándo paro?	Si no, …
`2 longitud_lista`	lista vacía → 0	1 + longitud del resto
`3 encontrar_minimo`	un elemento → ese	min(primero, mínimo del resto)
`4 posicion_de`	el primero es el buscado → 1	1 + posición en el resto
`5 encontrar_extremo`	un elemento → ese	funcion(primero, extremo del resto)
`6, 7 obtener_..._n`	n = 1 → el primero	buscar la posición n-1 en el resto
`8a duplicar_lista`	lista vacía → []	[2·primero] + resto duplicado
`9 repetir_accion_n`	n = 0 → no hago nada	hago la acción y repito con n-1

4. Ejercicio a ejercicio

Ejercicio 1 · imprimir_bienvenida

Un `print` con el texto; `print` ya añade el salto de línea al final. Es la única función que hace algo «hacia fuera» (escribe en pantalla); las demás solo calculan y devuelven un valor: son **funciones puras**.

```
def imprimir_bienvenida():
    print("¡Bienvenidos al Paradigma Funcional: Inmutabilidad y Funciones Puras!")
```

Ejercicio 2 · longitud_lista

Si la lista está vacía, mide 0. Si no, mide 1 (el primer elemento) + lo que mida el resto. No mira qué hay en la lista, así que vale para listas de cualquier tipo. `not lista` detecta la lista vacía sin llamar a `len`; si lo prefieres más explícito, `lista == []` hace lo mismo.

```
def longitud_lista(lista):
    if not lista:
        return 0
    return 1 + longitud_lista(lista[1:])
```

```
# longitud_lista([4, 8, 15])
# = 1 + longitud_lista([8, 15])
# = 1 + 1 + longitud_lista([15])
# = 1 + 1 + 1 + longitud_lista([])
# = 1 + 1 + 1 + 0 = 3
```

Ejercicio 3 · encontrar_minimo

Si solo queda un elemento, ese es el mínimo. Si no, me quedo con el menor entre el primero y el mínimo del resto, usando `min`. Ojo: aquí `min` solo compara dos números; quien recorre la lista es la recursión. (`min(lista)` lo haría todo de golpe, pero entonces no estarías resolviendo nada.)

```
def encontrar_minimo(lista):
    if not lista[1:]:
        return lista[0]
    return min(lista[0], encontrar_minimo(lista[1:]))
```

```
# encontrar_minimo([9, 3, 7])
# = min(9, encontrar_minimo([3, 7]))
# = min(9, min(3, encontrar_minimo([7])))    [7][1:] está vacío -> 7
# = min(9, min(3, 7))
# = min(9, 3) = 3
```

Ejercicio 4 · encontrar_minimo_y_posicion

Lo parto en dos problemas: **1)** el mínimo, que ya lo calcula `encontrar_minimo` (ejercicio 3); **2)** en qué posición está, con una función auxiliar `posicion_de`. Al final devuelvo los dos valores en una **tupla** `(minimo, posicion)`, que es justo lo que pide el enunciado.

`posicion_de` es casi igual que `longitud_lista`: suma 1 por cada elemento que pasa hasta encontrar el que busca. Si el mínimo está repetido, devuelve la primera aparición, porque para en cuanto lo encuentra. `minimo` se asigna una sola vez y no cambia nunca (no es mutación): solo evita calcular el mínimo dos veces.

```
def posicion_de(elemento, lista):
    if lista[0] == elemento:
        return 1
    return 1 + posicion_de(elemento, lista[1:])


def encontrar_minimo_y_posicion(lista):
    minimo = encontrar_minimo(lista)
    return (minimo, posicion_de(minimo, lista))
```

```
# encontrar_minimo_y_posicion([9, 1, 7, 3])
# 1) minimo = encontrar_minimo([9, 1, 7, 3]) = 1
# 2) posicion_de(1, [9, 1, 7, 3])        9 no es 1 -> 1 + ...
#    = 1 + posicion_de(1, [1, 7, 3])     1 es 1    -> 1
#    = 1 + 1 = 2
# resultado: (1, 2)
```

Ejercicio 5 · sumar_max_y_min (refactorización)

Parto de `encontrar_minimo`: el máximo se calcularía igual, cambiando `min` por `max`. Así que la copio, la renombro a `encontrar_extremo`, le añado un parámetro `funcion` y cambio `min` por `funcion`. Luego la llamo con `max` y con `min` y sumo. En Python no hace falta `#'` ni `funcall`: `max` se pasa por su nombre y `funcion(a, b)` la ejecuta. Lo único que cambia respecto al ejercicio 3:

```
ej. 3:  min    (lista[0], encontrar_minimo (lista[1:]))
ej. 5:  funcion(lista[0], encontrar_extremo(lista[1:], funcion))
```

```
def encontrar_extremo(lista, funcion):
    if not lista[1:]:
        return lista[0]
    return funcion(lista[0], encontrar_extremo(lista[1:], funcion))


def sumar_max_y_min(lista):
    return encontrar_extremo(lista, max) + encontrar_extremo(lista, min)
```

```
# sumar_max_y_min([2, 6, 10]) = 10 + 2 = 12
```

Ejercicio 6 · obtener_caracter_n

Avanzo por la lista restando 1 a `n`. Cuando `n` vale 1, el primer elemento es el que busco. En Python no hay tipo carácter: `'a'` es una cadena de un solo carácter, y el ejemplo del enunciado `['a','b','c','d']` se escribe tal cual. Si `n` es mayor que la lista, esta se acaba antes y `lista[0]` da error (`IndexError`); el enunciado no pide controlar ese caso.

```python
def obtener_caracter_n(lista, n):
    if n == 1:
        return lista[0]
    return obtener_caracter_n(lista[1:], n - 1)
```

```python
# obtener_caracter_n(['a', 'b', 'c', 'd'], 2)
# = obtener_caracter_n(['b', 'c', 'd'], 1)    n = 1 -> devuelvo el primero
# = 'b'
```

Ejercicio 7 · obtener_trinomio_n

Es el mismo código que el 6, cambiando solo el nombre. Cada trinomio es una tupla `(1, 2, 3)`. Ojo: el ejemplo del enunciado pone `obtener_caracter_n` por error; lo indico en un comentario.

¿Se puede generalizar a cualquier tipo? Sí: la función nunca mira qué hay dentro de cada elemento (solo avanza con `lista[1:]` y devuelve `lista[0]`) y Python tiene tipado dinámico, así que vale para números, caracteres, cadenas, trinomios… (**polimorfismo**). En el fichero lo pruebo con una lista de cadenas. Como extra, con anotaciones de tipo se puede dejar escrito que es genérica: `def obtener_elemento_n[T](lista: list[T], n: int) -> T` (T = «cualquier tipo»).

```python
def obtener_trinomio_n(lista, n):
    if n == 1:
        return lista[0]
    return obtener_trinomio_n(lista[1:], n - 1)
```

```python
# obtener_trinomio_n([(1, 2, 3), (4, 5, 6)], 2)     => (4, 5, 6)
# obtener_trinomio_n(["uno", "dos", "tres"], 3)     => 'tres'
```

Ejercicio 8 · duplicar_lista

a) Recursiva: el doble del primero, metido en una lista de un elemento y unido con `+` delante del resto ya duplicado. **b)** `map` con una lambda que duplica un número; como `map` devuelve un iterador, lo paso a lista con `list(...)`. La b) se llama `duplicar_lista_map` porque, si se llamara igual, la segunda definición sustituiría a la primera. En los dos casos se crea una lista nueva y la original no cambia.

```python
def duplicar_lista(lista):
    if not lista:
        return []
    return [2 * lista[0]] + duplicar_lista(lista[1:])


def duplicar_lista_map(lista):
    return list(map(lambda x: 2 * x, lista))
```

```python
# duplicar_lista([1, 2, 3])
# = [2] + duplicar_lista([2, 3])
# = [2] + [4] + duplicar_lista([3])
# = [2] + [4] + [6] + duplicar_lista([])
# = [2] + [4] + [6] + [] = [2, 4, 6]
#
# duplicar_lista_map([1, 2, 3])
# map aplica la lambda a 1, 2, 3 -> 2, 4, 6 -> list(...) = [2, 4, 6]
```

c) Una lambda es una función sin nombre que se escribe justo donde se usa (`lambda x: 2 * x`). En Python solo puede contener una expresión. Ventajas: se pasa directamente a funciones como `map` o `reduce`, evita crear con `def` funciones de un solo uso, puede usar variables del sitio donde se escribe (clausura) y el código dice *qué* hacer, no *cómo* recorrer la lista.

Ejercicio 9 · repetir_accion_n

Si `n > 0`: ejecuto la acción con `accion()` y me llamo con `n - 1`. Cuando `n` llega a 0, no entra en el `if` y la función termina. Las dos líneas de dentro del `if` se ejecutan en orden (en LISP hacía falta `when` para eso).

La demostración va directamente en el fichero (sin función): pregunta un número con `input()`, lo convierte con `int()` y pasa la bienvenida como `imprimir_bienvenida`, **sin paréntesis**, para pasar la función y no ejecutarla. Python limita la recursión a unas 1000 llamadas; con números normales no hay problema.

```python
def repetir_accion_n(accion, n):
    if n > 0:
        accion()
        repetir_accion_n(accion, n - 1)


veces = int(input("\n¿Cuántas veces quieres mostrar la bienvenida? "))
repetir_accion_n(imprimir_bienvenida, veces)
```

```python
# con n = 2:  n=2 -> imprime   n=1 -> imprime   n=0 -> para
```

Ejercicio 10 · map/reduce y el lenguaje

Incluyo un ejemplo pequeño: cada sublista es un bloque de datos que estaría en un nodo; `map` cuenta cada bloque (con `longitud_lista`) y `reduce` suma esos resultados de dos en dos. `reduce` hay que importarlo al principio del fichero con `from functools import reduce`.

```
def contar_elementos_map_reduce(bloques):
    return reduce(lambda a, b: a + b, map(longitud_lista, bloques))
```

```
# map(longitud_lista, [[1, 2, 3], [4, 5], [6, 7, 8, 9]])   ->  3, 2, 4
# reduce(lambda a, b: a + b, [3, 2, 4])  =  (3 + 2) + 4    ->  9
```

Ideas clave de las respuestas (desarrolladas en el fichero):

- **a)** Un `for` va dato a dato y cambia una variable (el acumulador), así que tiene que hacerse en orden y en un solo sitio. Con `map` cada dato se procesa por separado, así que se puede repartir entre muchos nodos a la vez; `reduce` junta los resultados parciales. Ventajas: escala, no hay variables compartidas (ni conflictos entre nodos), si un nodo falla se repite su parte en otro con el mismo resultado, y el código dice *qué* hacer, no *cómo*.
- **b)** Python: multiparadigma (imperativo, orientado a objetos y funcional). Lo funcional: funciones como dato, lambdas, recursión, clausuras, `map`/`filter`/`reduce`, tuplas y cadenas inmutables. **No es funcional puro**: deja modificar variables y listas (la inmutabilidad depende de ti), no optimiza la recursión de cola y limita la recursión a ~1000 llamadas. Interpretado, bloques por sangría, memoria automática y REPL. Tipado **dinámico** (los tipos se comprueban al ejecutar, «duck typing») y **fuerte** (`1 + "a"` da `TypeError`), con anotaciones de tipo opcionales que revisan herramientas como mypy. Implica más flexibilidad (una función vale para cualquier tipo), a cambio de descubrir los errores de tipo al ejecutar.
- **Por qué los lenguajes modernos lo incorporan:** ordenadores con varios núcleos y sistemas distribuidos (sin datos que cambien, el paralelismo es más seguro), código más corto con map/filter/reduce y lambdas, funciones puras fáciles de probar, y encaja con big data.

5. Antes de entregar

- Nombre del fichero exacto: `angel_panadero_rodriguez_AEC1.py`.
- Cabecera con `# Paradigmas de la Programación`, `# AEC1` y `# Nombre: ...`, y un comentario `# Ejercicio N` antes de cada ejercicio. El enunciado usa `;` porque es el comentario de LISP; en Python es `#`.
- Nombres de funciones idénticos a los del enunciado, con guion bajo.
- Respuestas a las preguntas (7, 8c, 10a, 10b) como comentarios dentro del fichero.
- Busca en tu fichero (Ctrl+F) `for`, `while`, `len` y `append`: solo deberían aparecer dentro de comentarios.
- Sangría siempre con 4 espacios (no mezcles tabuladores).
- Ejecútalo una vez con `python angel_panadero_rodriguez_AEC1.py` y comprueba que no da errores.